

SRGGE Project — Scalable Rendering of a Virtual Museum

Alessandro Fazio

1. Introduction

This project implements a scalable real-time renderer for a tile-based virtual museum populated with high-resolution 3D models (Armadillo, Bunny, Dragon, Happy Buddha). The primary engineering goal is to maintain an interactive, stable frame rate while navigating an environment whose models, at full resolution, total tens of millions of triangles. To achieve this, the engine relies on a multi-tiered optimization architecture:

1. Level-of-Detail (LOD) generation: offline decimation via Quadric Error Metrics (QEM) and normal clustering.
2. Time-critical rendering: a greedy LOD selector guided by a per-frame triangle budget and temporal hysteresis.
3. Spatial culling: a Potentially Visible Set (PVS) precomputed offline using deterministic supercover raycasting to cull occluded geometry at runtime.

All measurements reported here were taken on an NVIDIA GeForce RTX 5050 Laptop GPU at a 800×600 viewport.

2. System Architecture

The engine is structured around a decoupled architecture, separating geometry storage from spatial placement. The `TriangleMesh` class parses and stores the GPU buffers (VAO/VBOs) once per model, while `TriangleMeshInstance` objects represent the physical entities in the room, holding lightweight transformation matrices and tracking their own LOD state.

3. LOD Generation — Topologically Watertight Simplification (Sessions 2–4)

To reduce the geometry load, the engine precomputes three simplified LODs per model using a sparse regular grid (`std::map`) and, per cell, the Quadric Error Metric to find the optimal collapse vertex. For each triangle the supporting plane is expressed relative to the cell centre (for numerical stability on very large models) and accumulated into a 4×4 quadric. The optimal representative is the minimum of that quadric, solved with the Jacobi SVD pseudo-inverse from the Eigen library — this handles the frequent rank-deficient (planar) neighbourhoods that a direct inverse cannot. The grid resolutions are 100 / 50 / 35 cells along the longest axis. For the Dragon the triangle count drops across LODs from 7,218,906 to 129,582 / 73,650 / 60,572.

3.1 The “Swiss cheese” topology problem

Initially the simplification used face normals to assign vertices to the 8 sub-nodes (octants) of each grid cell. But a single vertex on a cell boundary is shared by faces pointing in very different directions, so the logic split shared vertices into different sub-nodes — literally tearing the geometry and opening visible holes (the “Swiss cheese” effect) at lower resolutions.

3.2 The fix: per-vertex normals

The fix was an analytical “Pass 0” before building the quadrics: the engine computes a weighted average of adjacent face normals to obtain a unified vertex normal, and uses that to assign the sub-node. The spatial index then becomes consistent for shared vertices, and the resulting meshes stay watertight across LOD levels. At the most aggressive level (LOD 3) the vertex-clustering produces the characteristic large, faceted triangles; this is the expected signature of the technique and is only visible up close, whereas LOD 3 is used when the model is far and small on screen.

3.3 Selectable clustering modes (runtime comparison)

All three simplification techniques are implemented and can be switched at runtime with the M key, which regenerates the LODs of every model with the chosen method (with a short pause while the Dragon is reprocessed). This makes the trade-off between them directly observable on the same scene, as required by Session 4:

Technique	Cell sub-nodes	Dragon LOD 3	Representative vertex
Simple vertex clustering	1	3,618	Average of cell vertex positions
QEM (standard)	1	3,618	Quadric minimum, via SVD
QEM + normal clustering	up to 8	60,572	Quadric minimum, split per normal

Table 1. The three clustering modes on the same models. Simple and standard QEM keep one representative per cell, so they produce the same triangle count; QEM places that vertex to minimise geometric error, giving crisper silhouettes than the simple average. Normal clustering splits each cell by surface orientation, preserving thin or double-sided features (the Bunny’s ears, the Dragon’s legs) at the cost of more triangles.

Visually, simple clustering rounds off and slightly inflates sharp regions because the average pulls vertices toward cell centres; standard QEM keeps edges and corners sharp at the same triangle budget; and normal clustering is the only one of the three that does not merge opposite-facing surfaces, which is why it alone stays free of the pinching seen on thin features at the most aggressive level.

4. Time-Critical Rendering & Bottleneck Resolution (Session 5)

The greedy algorithm dynamically assigns LODs by maximising the heuristic score $\Delta\text{benefit} / \Delta\text{cost}$, starting every object at the lowest LOD and promoting the best candidate until the per-frame triangle budget is exhausted. Early testing, however, showed the models locked at the lowest LOD and the frame rate artificially capped.

4.1 The hidden $O(n)$ in the hot loop

The most damaging bottleneck was the auto-LOD path itself: it ran at less than 30 FPS while the same scene at manual LOD 0 — at least 10 times more triangles — ran at a steady 60 FPS. Since more geometry was faster, the cost was not on the GPU. The cause was `getDiagonal()`, which recomputed each model’s bounding box over all of its original vertices (millions, for the Dragon) on every call, and the greedy loop called it once per object per frame. Caching the diagonal (it never changes, as simplification only edits a working copy) turned the call into $O(1)$ and the auto-LOD path immediately recovered to a stable 60 FPS. This single fix, combined with a fixed budget, removed all of the auto-LOD frame-rate problems.

4.2 The “ghost cost” budget bleeding

Even once the frame rate was unlocked (unlocked just for testing), the greedy algorithm initially failed to upgrade models because the per-frame reset added the base cost of every model —

including those hidden behind walls — to the budget. Filtering the reset to count only objects validated by the PVS freed the budget to focus on the visible scene, after which the nearest objects correctly received the highest detail. The budget itself follows the course formula $\text{MaxCost} = \text{TPS} / \text{FPS}$, a fixed value derived from the triangle throughput the GPU sustains divided by the target frame rate (a constant, not a per-frame feedback loop).

A toggle (C) enables LOD colouring (white = LOD 0, blue = 1, yellow = 2, red = 3), used to verify the selection: in Figure 1 the near Bunny is white (LOD 0) distant models fall to blue and yellow while the unoptimised pieces stay red, demonstrating that the greedy spends its budget on the closest objects.

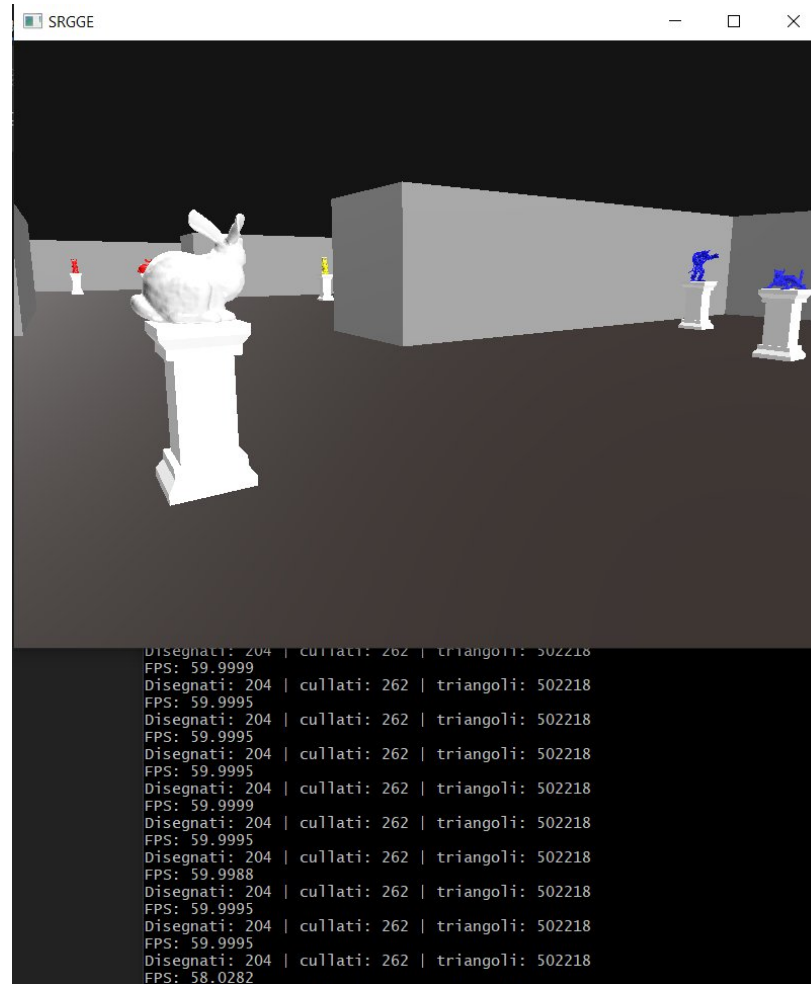


Figure 1. LOD colouring with the greedy selector active. The near Bunny is white (LOD 0) and the more distant Armadillo/Dragon are blue/yellow;

5. Temporal Hysteresis (Session 6)

A visual artefact known as LOD thrashing occurs when the camera hovers exactly at the threshold distance between two LOD levels: the score fluctuates every frame, swapping geometry and colour dozens of times per second. To cure this the engine applies frame-delay hysteresis: when a free object changes LOD it is locked for $N = 20$ frames, during which it is excluded from the greedy loop and simply deducts its committed cost from the budget.

Entering the greedy with locked objects already fixed (as the specification requires) is more faithful than clamping after the fact. The result crystallises the transitions and absorbs the micro-fluctuations of the camera; the trade-off is that the rendered triangle count may transiently exceed the nominal budget while objects are locked.

6. Offline PVS Culling (Session 7)

Submitting all of the museum's statues to the GPU simultaneously is wasteful when most are occluded. A separate, non-OpenGL C++ tool parses the layout (level.txt) and precomputes cell-to-cell visibility on the 2D occupancy grid.

The tool implements the standard DDA voxel-traversal algorithm — the same approach the course reference presents as the “Bresenham supercover” — driven by a per-axis unit step size (rayUnitStepSize) and a sign-based step vector, advancing to whichever axis boundary is nearer at each step. Explicit handling of exact corner crossings turns it into a true supercover: every grid cell the ray touches, corners included, is evaluated, so visibility cannot leak diagonally through the corners of walls. Cell-to-cell visibility uses deterministic 5-point sampling (centre plus four inset corners); if any ray reaches the target without crossing a wall, the cells are mutually visible. Because the test ORs over multiple rays, the PVS is conservative — it never culls something that is actually visible. The resulting table (pvs.txt) is loaded once at start-up, giving the render loop an O(1) visibility lookup. Culling is applied to both models and floor tiles. Since visibility is purely 2D, the camera may ascend/descend (Q/E) without changing the visible set, as required.

The map used for the benchmarks is a 20×15 grid (300 cells: 180 navigable, 120 wall) arranged as four corner rooms around a central cross of corridors, with a solid block of wall cells at the centre acting as the main occluder. The tool computes all 16,110 navigable-cell pairs in well under a second; on average a cell sees 56 others. Figure 2 shows the visibility from the spawn cell: the camera sees its own quadrant and the connecting corridors, but the central block hides the far quadrants.

```
#####
#..VVVV#.#.....#
#...VVV#.#.....#
#...VVV#.#.....#
#####VVV#VVVV#####
#VVVVVVVVVVVVVVV#
#VVVCV#####.....#
###VVV#####.....###
#VVVVV#####.....#
#VVVVV.....#
#####VVV#.#.....#
#...VVV#.#.....#
#...VVV#.#.....#
#...VVV#.#.....#
#####
```

Figure 2. PVS visibility from the spawn cell (C). V = visible, . = culled, # = wall. The camera sees its own room and the central corridors; the solid block in the middle occludes the opposite quadrants, which are culled.

7. Results & Bottleneck Analysis

7.1 Effect of LOD and PVS

The configurations below were measured live while navigating the museum (numbers read directly from the on-screen counters):

Config	PVS	LOD	FPS	Objects drawn	Model triangles
Auto LOD, no PVS	off	greedy	60	466	666,296

Auto LOD + PVS	on	greedy	60	~200	469,308
Manual LOD 0 + PVS	on	LOD 0	60	167	15,944,082
Manual LOD 1, no PVS	off	LOD 1	60	466	1,748,598

Two independent reductions are visible. The greedy LOD selector cuts the visible-set geometry from 15.9 M triangles (manual LOD 0) to 0.47 M (a 34× reduction) at the same 60 FPS. PVS removes roughly 57 % of the drawn objects (466 → ~200) and a further slice of triangles by discarding the statues behind the central wall. On this GPU both techniques hold a steady 60 FPS even at 16 M triangles, so here they buy headroom rather than a raw frame-rate gain — that headroom is what keeps the system at target on weaker hardware or in denser scenes, and is precisely the point of a triangle budget. The fully unoptimised case (all models at LOD 0 with culling off, tens of millions of triangles) is the worst case the pipeline is designed to avoid.

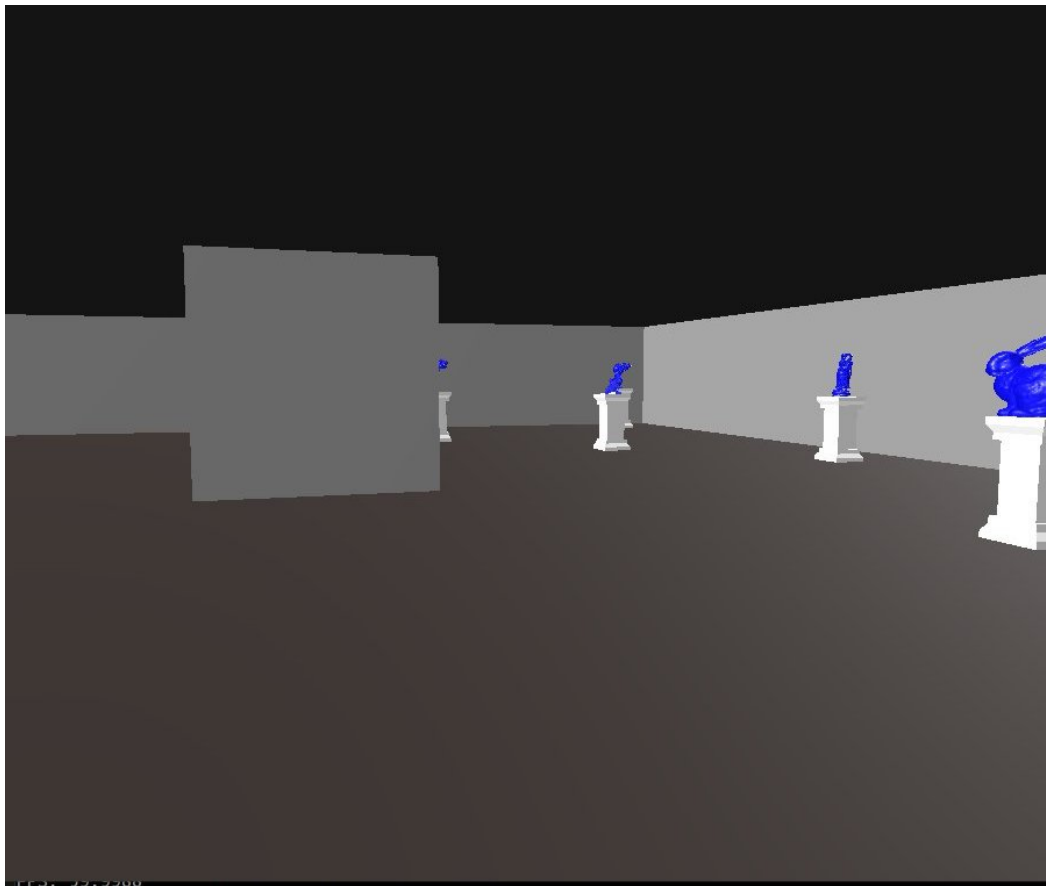


Figure 3. The same museum with PVS disabled (console: `cullati = 0`, all 466 objects drawn). The solid grey block in the centre is the occluder; from this viewpoint the statues on its far side are still submitted to the GPU, which is exactly the work PVS removes when enabled.

7.2 Geometry-bound vs fill-rate-bound

Profiling shows the bottleneck shifts with context. Down a corridor facing several statues the engine is geometry-bound — the cost is vertex processing and bus transfer — which is the regime the greedy LOD selector targets. Conversely, with the camera extremely close to a single model that fills the screen, the frame rate can dip even though the triangle count falls, because the cost moves to the fragment shader and overdraw (fill-rate-bound); LOD and PVS cannot help that

regime. To mitigate overdraw the render queue is sorted front-to-back so the depth test (early-Z) rejects hidden fragments before the fragment shader runs, which helps when objects occlude one another but not when a single object dominates the screen. Identifying the two regimes by watching whether triangle counts rise or fall during a drop is itself the key observation.

8. Implementation Choices and Deviations from the Specification

- Map format. The map is split into map.txt (model paths) and level.txt (a digit grid encoding wall/floor/model-per-cell), instead of the single “W H” + grid + explicit-instance-transform format shown in the slides.
- Corner model. Walls are built from axis-aligned scaled cubes, so corner cells are filled automatically; a dedicated Corner model is redundant in this design.
- Deterministic sampling. Visibility uses fixed sample points (centre + four inset corners) rather than random sampling, for reproducible results.

9. Conclusions

The renderer implements the full scalable-rendering pipeline: QEM and normal-clustering LOD generation that stays topologically watertight, a time-critical greedy selector with temporal hysteresis, and an offline conservative PVS used for runtime culling. The optimisations reduce the submitted geometry by an order of magnitude (the Dragon alone from 7.2 M to ~60 K triangles) and remove most occluded objects, sustaining a stable 60 FPS in every interactive configuration. The decisive performance lesson was that the worst stall was not on the GPU at all but a per-frame $O(n)$ bounding-box recomputation in the LOD loop; once cached, the time-critical system behaved exactly as intended.

10. Controls (reference)

- F1 — toggle navigation mode
- W A S D — move;
- L — toggle automatic (greedy) LOD vs manual
Ctrl + / Ctrl - — manual LOD level
- C — toggle LOD colouring
- P — toggle PVS culling
- M — cycle clustering mode (simple vertex / QEM / QEM + normal)

start :
./SRGGE.exe ../map.txt